

1. Write a program to perform the following

- o An empty list
- o A list with one element
- o A list with all identical elements
- o A list with negative numbers

Test Cases:

1. Input: []
o Expected Output: []
2. Input: [1]
o Expected Output: [1]
3. Input: [7, 7, 7, 7]
o Expected Output: [7, 7, 7, 7]
4. Input: [-5, -1, -3, -2, -4]
o Expected Output: [-5, -4, -3, -2, -1]

The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python program that defines a `sort_list` function and tests it with four cases. The output panel on the right shows the results of these tests, which match the expected outputs from the test cases. The code is as follows:

```
1- def sort_list(arr):
2-     return sorted(arr)
3
4- # Test Cases
5- test_cases = [
6-     [],
7-     [1],
8-     [7, 7, 7, 7],
9-     [-5, -1, -3, -2, -4]
10 ]
11
12- for i, test in enumerate(test_cases, 1):
13-     print(f"Test Case {i}:")
14-     print("Input:", test)
15-     print("Output:", sort_list(test))
16-     print()
```

The output panel displays the following results:

```
Test Case 1:
Input: []
Output: []

Test Case 2:
Input: [1]
Output: [1]

Test Case 3:
Input: [7, 7, 7, 7]
Output: [7, 7, 7, 7]

Test Case 4:
Input: [-5, -1, -3, -2, -4]
Output: [-5, -4, -3, -2, -1]

=== Code Execution Successful ===
```

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

Sorting a Random Array:

Input: [5, 2, 9, 1, 5, 6]

Output: [1, 2, 5, 5, 6, 9]

Sorting a Reverse Sorted Array:

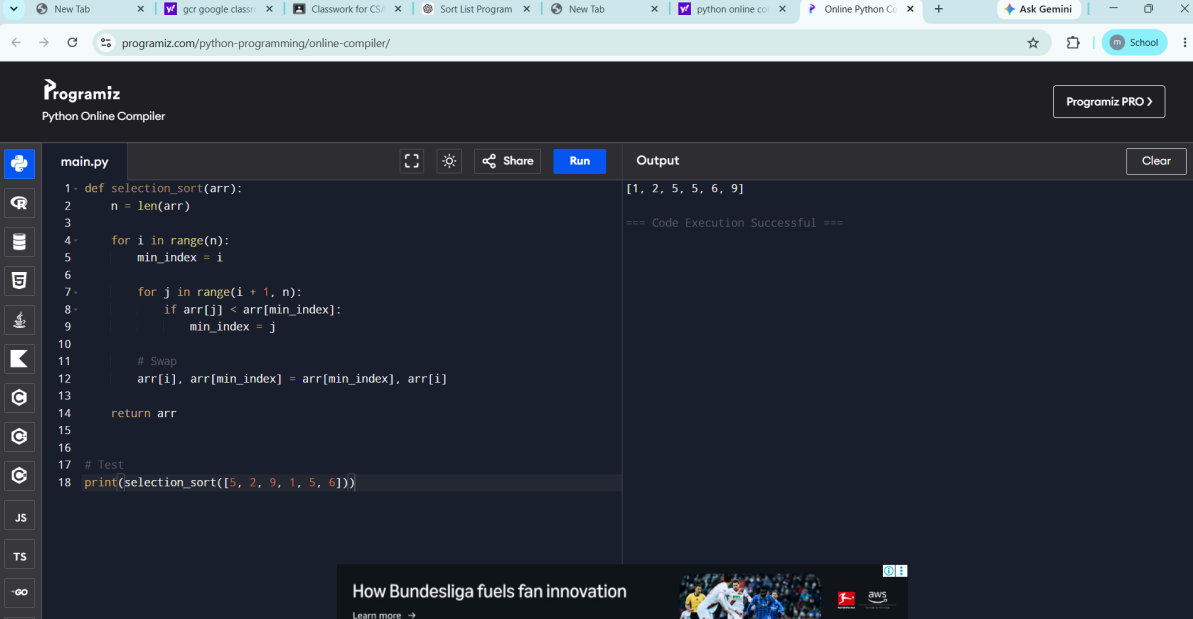
Input: [10, 8, 6, 4, 2]

Output: [2, 4, 6, 8, 10]

Sorting an Already Sorted Array:

Input: [1, 2, 3, 4, 5]

Output: [1, 2, 3, 4, 5]

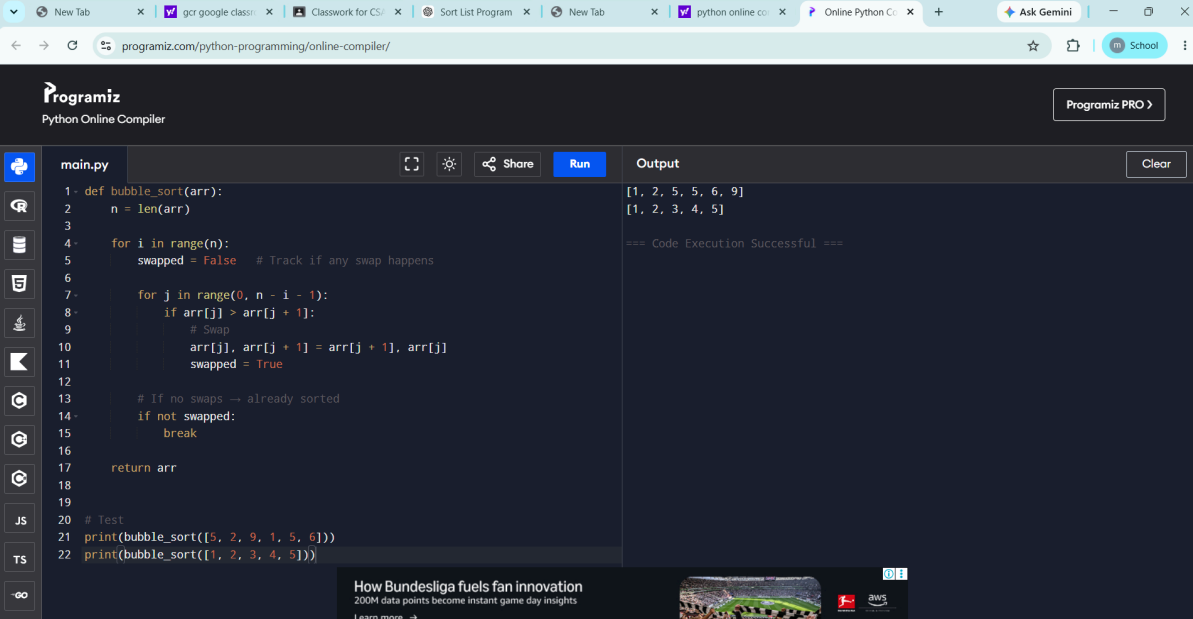


The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for a selection sort function. The output panel on the right shows the result of the execution.

```
main.py
1- def selection_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         min_index = i
6-
7-         for j in range(i + 1, n):
8-             if arr[j] < arr[min_index]:
9-                 min_index = j
10-
11-         # Swap
12-         arr[i], arr[min_index] = arr[min_index], arr[i]
13-
14-     return arr
15-
16- # Test
17- print(selection_sort([5, 2, 9, 1, 5, 6]))
```

Output: [1, 2, 5, 5, 6, 9]
=== Code Execution Successful ===

3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.



The screenshot shows the Programiz Python Online Compiler interface with a modified bubble sort function. The code editor on the left contains the Python script. The output panel on the right shows the results of the execution.

```
main.py
1- def bubble_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         swapped = False # Track if any swap happens
6-
7-         for j in range(0, n - i - 1):
8-             if arr[j] > arr[j + 1]:
9-                 # Swap
10-                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
11-                 swapped = True
12-
13-         # If no swaps → already sorted
14-         if not swapped:
15-             break
16-
17-     return arr
18-
19- # Test
20- print(bubble_sort([5, 2, 9, 1, 5, 6]))
21- print(bubble_sort([1, 2, 3, 4, 5]))
```

Output: [1, 2, 5, 5, 6, 9]
[1, 2, 3, 4, 5]
=== Code Execution Successful ===

4. Test Cases:

Test your optimized function with the following lists:

1. Input: [64, 25, 12, 22, 11]

Expected Output: [11, 12, 22, 25, 64]

2. Input: [29, 10, 14, 37, 13]

Expected Output: [10, 13, 14, 29, 37]

3. Input: [3, 5, 2, 1, 4]

Expected Output: [1, 2, 3, 4, 5]

4. Input: [1, 2, 3, 4, 5] (Already sorted list)

Expected Output: [1, 2, 3, 4, 5]

The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for bubble sort. The output panel on the right displays the results of four test cases. The code is as follows:

```
main.py
7- for j in range(0, n - i - 1):
8-     if arr[j] > arr[j + 1]:
9-         # Swap
10-        arr[j], arr[j + 1] = arr[j + 1], arr[j]
11-        swapped = True
12-
13-    # Stop early if no swaps
14-    if not swapped:
15-        break
16-
17-    return arr
18-
19-
20- # Test Cases
21- test_cases = [
22-     [64, 25, 12, 22, 11],
23-     [29, 10, 14, 37, 13],
24-     [3, 5, 2, 1, 4],
25-     [1, 2, 3, 4, 5]
26- ]
27-
28- for i, test in enumerate(test_cases, 1):
29-     print(f"Test Case {i}:")
30-     print("Input :", test)
31-     print("Output:", bubble_sort(test))
32-     print()
```

The output panel shows the following results:

```
Test Case 1:
Input : [64, 25, 12, 22, 11]
Output: [11, 12, 22, 25, 64]

Test Case 2:
Input : [29, 10, 14, 37, 13]
Output: [10, 13, 14, 29, 37]

Test Case 3:
Input : [3, 5, 2, 1, 4]
Output: [1, 2, 3, 4, 5]

Test Case 4:
Input : [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

--- Code Execution Successful ---
```

5. Given an array arr of positive integers sorted in a strictly increasing order, and an integer k. return

the kth positive integer that is missing from this array.

Example 1:

Input: arr = [2,3,4,7,11], k = 5

Output: 9

Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...]. The 5th missing positive integer is 9.

Example 2:

Input: arr = [1,2,3,4], k = 2

Output: 6

Explanation: The missing positive integers are [5,6,7,...]. The 2nd missing positive integer is 6.

```
1- def findKthPositive(arr, k):
2-     for i in range(len(arr)):
3-         missing = arr[i] - (i + 1)
4-
5-         if missing >= k:
6-             return i + k
7-
8-     # If kth missing is beyond last element
9-     return len(arr) + k
10
11
12 # Test Cases
13 print(findKthPositive([2,3,4,7,11], 5)) # Output: 9
14 print(findKthPositive([1,2,3,4], 2))    # Output: 6
```

6. A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return

the index to any of the peaks. You may imagine that `nums[-1] = nums[n] = -∞`. In other words,

an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 2

Explanation: 3 is a peak element and your function should return the index number 2.

Example 2:

Input: `nums = [1,2,1,3,5,6,4]`

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` defines a function `findPeakElement` that uses a binary search approach to find a peak element in a list of numbers. The function returns the index of the peak element. Test cases are provided at the bottom of the code block.

```
1- def findPeakElement(nums):
2-     low, high = 0, len(nums) - 1
3-
4-     while low < high:
5-         mid = (low + high) // 2
6-
7-         if nums[mid] < nums[mid + 1]:
8-             low = mid + 1 # move right
9-         else:
10-            high = mid # move left
11-
12-     return low
13-
14-
15- # Test Cases
16- print(findPeakElement([1, 2, 3, 1])) # Output: 2
17- print(findPeakElement([1, 2, 1, 3, 5, 6, 4])) # Output: 5 or 1
```

The output on the right shows the results of the test cases: 2 and 5, followed by the message "=== Code Execution Successful ===".

7. Given two strings needle and haystack, return the index of the first occurrence of needle in

haystack, or -1 if needle is not part of haystack.

Example 1:

Input: haystack = "sadbutsad", needle = "sad"

Output: 0

Explanation: "sad" occurs at index 0 and 6.

The first occurrence is at index 0, so we return 0.

Example 2:

Input: haystack = "leetcode", needle = "leeto"

Output: -1

Explanation: "leeto" did not occur in "leetcode", so we return -1.

The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` defines a function `strStr` that finds the index of the first occurrence of a needle in a haystack. The function returns the index of the first occurrence, or -1 if the needle is not found. Test cases are provided at the bottom of the code block.

```
1- def strStr(haystack, needle):
2-     n, m = len(haystack), len(needle)
3-
4-     # Edge case
5-     if m == 0:
6-         return 0
7-
8-     for i in range(n - m + 1):
9-         if haystack[i:i+m] == needle:
10-            return i
11-
12-     return -1
13-
14-
15- # Test Cases
16- print(strStr("sadbutsad", "sad")) # Output: 0
17- print(strStr("leetcode", "leeto")) # Output: -1
```

The output on the right shows the results of the test cases: 0 and -1, followed by the message "=== Code Execution Successful ===".